

Document number: P2784R0
Date: 2023-02-09
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Not halting the program after detected contract violation

This paper explores the possibility of not stopping the program after a contract violation has been detected at run-time. This is a feature to be added to the contract support framework.

1. Motivation

{mot}

Contract annotations are tools for expressing what constitutes a program that is running against its specification ("specification" understood as information provided in contract annotations). If a runtime check based on such annotation returns `false`, we can be sure that this is happening right now: the program runs against the specified intentions. If a program is let to continue at this point, one of the likely consequences is that it will either crash or start behaving in an unpredictable way (e.g. by hitting an undefined behavior at the language level):

```
int * p = 0;
[[assert: p != 0]];
return *p > 3; // bad things!
```

For this reason papers like [\[P2388R4\]](#) suggest to immediately halt the program at this point which, while being a harsh reaction, gives a guaranteed repeatable result with an upper limit on the possible consequences.

While this seems a good default, there are situations where stopping the program this way is not the optimum solution. These include:

1. When there is an isolated subcomponent in a program, maybe relatively new, where we can be fairly confident that a bug in the subcomponent will not affect the correctness of the remaining parts of the program.
2. A special case of the above is when we consider function `main()` as such submodule. We may know that a bug in function `main()` may not affect the correctness of the second call to `main()` if we somehow managed to restart it from within the program.
3. In case of unit tests, we may deliberately violate the function contract in order to determine the existence of safety measures such as contract annotations.
4. When the program has been running in production for a long while, and likely taken all the possible control paths, we may be fairly confident that even if the program does not stick to its internal specification, it still behaves according to user expectations.

The first three cases have one thing in common: while we do not want the program to halt, it is allowed, and even desired, to resume the program execution from a different place than where the contract violation was observed. In this paper we only focus on these use cases. This sounds like a job for the exception handling mechanism. We consider it as one of the options, however this paper also proposes another.

2. Exceptions

{exc}

The problem of transferring control to a different location upon "failure" is addressed in C++ by the exception handling mechanism. This mechanism blends nicely with other parts of the language by:

- releasing all acquired resources by invoking class destructors,
- allowing the non-failure control path to be expressed cleanly, without any visible `if`-statements.

This is why [\[P0542R5\]](#) allowed a "mode" where upon a detected contract violation a programmer-installed violaiton handler is invoked, and this handler as one of possible opitons can throw an exception. This way we get a two-fold guarantee:

- the code following the contract annotation is not executed (due to exception),
- control is transfered to a different place, where hopefully the program can recover.

This is also what [\[P2698R0\]](#) proposes under a new translation mode **Eval_and_throw**.

Using exceptions to recover from contract violations, however, is problematic for a number of reasons.

The first issue is conceptual. Exceptions were meant to handle the situation where a **correct** program responds in an exceptional way to an exceptional situation. This exceptional recovery still executes some code (primarily destructors) and the executed code also assumes that the program is correct and it has its own preconditions.

Consider a class with an invariant expressed in the source code via preconditions.

```
class State
{
    std::vector<Column*> _columns;
    unsigned _theColumn;

public:
    bool invariant() const noexcept
    {
        return _theColumn < _columns.size() && _columns[_theColumn] != nullptr;
    }

    void alter()
        [[pre: invariant()]]
        [[post: invariant()]];
}
```

```

~State()
[[pre: invariant()]]
{
    delete _columns[_theColumn];
}
};

```

If upon calling `state.alter()` its precondition is violated and this gets turned into an exception, during the stack unwinding we will need to call the destructor of `state`. The destructor also has a precondition which would also be violated. This would trigger a second exception to be thrown, which in C++ normally results in calling `std::terminate`.

We could put it in another way: destructors also have preconditions, and they may call other functions with contract annotations. If any of these checks fail, we get a throwing destructor, which likely aborts the program.

The second issue is more pragmatic: the interaction with `noexcept` functions, which can also have contract annotations. If contract checks are allowed to throw, we have a new question to answer, that [P2388R4](#) doesn't have to answer: are contract conditions evaluated inside or outside the function — because now it becomes a visible property:

```

void fun() noexcept [[pre: false]];

void test()
{
    fun(); // ok if precondition evaluated outside the function
          // std::terminate() if precondition evaluated inside the function
}

```

Note that even if we said that the precondition must be evaluated outside the function, this does not solve the problem of reporting contract violation via throw from nested `noexcept` functions:

```

void fun() noexcept [[pre: false]];

void gun() noexcept
{
    fun();
}

void test()
{
    gun(); // std::terminate()

    void(*pf)() noexcept = &fun;
    pf(); // std::terminate()
}

```

And in fact the issue is bigger than just `noexcept` functions. What should operator `noexcept` return?

```

void fun() noexcept [[pre: false]];

constexpr bool mystery = noexcept(fun());

```

Remember that operator `noexcept` tests full expressions along with invisible things like conversions and destructors. Should its value be dependent on the translation mode?

The scope of the problem is even wider. It is not limited to functions declared `noexcept` but also to functions that provide a no-fail guarantee, even if this is not statically checkable. This is how exception-safety (or failuer-safety) guarantees work: you can only provide a strong (commit or rollback) guarantee when you know that some operations never throw. When upon contract violation they nonetheless start to throw, no function can provide the declared level of exception safety. One could argue that if a precondition is violated, by definition, no guarantee is provided. But on the other hand, one of the goals of contracts is to offer some guarantees even if the contract is violated.

The third issue stems from the fact that a huge fraction of C++ programs is compiled with exceptions disabled. Yet, these programs have the same problem to solve: how not to halt the program upon contract violation.

3. Aborting a component

{abo}

As a potential solution to the above problem we propose a mechanism that is harsher than stack unwinding, but softer than `std::abort()`. In fact we are proposing a stricter and simpler version of the `setjmp/longjmp` mechanism. It is composed of two proposed Standard Library functions with special powers:

```

template <invokable F>
void abortable_component(F&& f);

```

This invokes the passed function `f` in almost a regular way, except that it is treated as "being executed in a separate component" as described below. Function `std::abortable_component` is exception-neutral: whatever the evaluation of `f()` throws is thrown out of function `std::abortable_component`. The goal of this function is to instruct the compiler what the programmer considers a component boundary.

```

[[noreturn]] void abort_component() noexcept;

```

Calling this function initiates the process of leaving the function call stack, without calling destructors of automatic objects and function parameters, until a component boundary, indicated by `std::abortable_component` is reached. Then the program resumes just after the call to

`std::abortable_component`. If no component boundary is found in the call stack then `std::abort` is called. Example:

```
struct Guard {
    ~Guard() { std::printf("A"); }
};

int fun() {
    Guard g;
    std::abortable_component();           // (2) abort sequence starts
    std::printf("B");                     // (3) this is skipped, "B" is never printed
}                                         // (4) destructor is skipped, "A" is never printed

int main() {
    std::abortable_component(&fun);       // (1) launching `fun` as a subcomponent
    std::printf("C");                     // (5) getting out of subcomponent, "C" is printed
}
```

We could say that this mechanism is similar to stack unwinding except that:

- there is no exception type matching, as there are no exception objects involved;
- no destructors — or other functions, such as copying exception objects — are invoked.

Alternatively, we could say that this is like `setjmp/longjmp`, except that the proposed approach is more structured: subcomponents have to nest. There is also no way to convey any information other than the fact that we are aborting. And there is no undefined behavior related to skipping destructors.

4. Discussion

{dis}

As one can easily observe, aborting a component can easily cause resource leaks, as destructors of automatic objects and function parameters are not executed. It may be more than just leaks. Not calling the destructor of `scoped_lock` can cause concurrency issues in other parts of the program. However, we are talking about a tool for minimizing damage in a desperate situation:

- a program cannot afford to halt,
- a program has a proof that it behaves incorrectly.

The goal is no longer to get everything right, but to minimize damage. Continuing without cleanup can cause further bugs, but so can continuing after a detected bug. This will be a dangerous feature, not recommended to be used for other situations.

Putting `std::abortable_component` in the program would mean that the programmer considers it reasonably safe to continue the execution of the program from that point even if arbitrary parts of the called function were skipped. This could be an option when invoking a "plugin" that is an optional part of the program, experimental and not required to fulfil the main program task.

The other part of the interface — function `std::abort_component()` would not even have to be exposed to programmers. For the purpose of the contract support framework it would be enough to say that in **Eval_and_abort** translation mode, when the predicate evaluates to `false` the effect is as if `std::abort_component()` was called. We wouldn't need a third translation mode, the effect of not aborting would be achieved by putting `std::abortable_component` in your program. Thus, when you want your program not to abort upon contract violation, you have to indicate a place (or places) where it is safe to resume the program from.

This feature can be introduced after the MVP in a backward-compatible fashion. For the MVP we can simply say that upon contract violation `std::abort()` is called. Then, after the MVP change it to calling `std::abort_component()`, which is indistinguishable from `std::abort()` as long as you have no call to `std::abortable_component()` in the program.

Finally, the proposed interface is very modest: there is no information conveyed about the point of and the reason for calling `std::component_abort()`. There is not information whether we returned normally or via the abort from `std::abortable_component()`. The mechanism could be extended to satisfy these expectations. However, the goal of this paper is to show the main idea behind the feature. Similarly, if this mechanism is used to handle contract violation, it could be combined with logging the information about the point of failure before calling `std::abort_component()`.

4.1. Use cases

{dis.use}

The use cases serviced by this feature.

4.1.1 Plugins

{dis.use.plu}

Suppose function `solve()` evaluates one of the user-provided plugins. it may have a bug, but if it fails — even if it leaked some resources — the rest of the program, or other user plugins may still work fine.

```
int solve(); // user plugin

bool call_user_plugin = true; // my flag for protecting against calling plugin twice, if it is proven buggy

std::optional<int> fun()
{
    std::optional<int> solution;

    if (call_user_plugin) {
        std::abortable_component([&]{ solution = solve(); });

        if (!solution)
```

```

        call_user_plugin = false;
    }

    return solution;
}

```

4.1.2. Killing a single thread

{dis.use.thr}

Suppose that you run a plugin in a separate thread, and in case of failure, you only want to kill that thread.

```

int solve(); // user plugin

int main()
{
    std::optional<int> solution;
    std::jthread t {[&]{
        try {
            std::abortable_component([&]{
                solution = solve();
            });
        }
        catch(...) {
            // swallow
        }
    }};

    // main thread ...
}

```

4.1.3. Unit-testing preconditions

{dis.use.utf}

Suppose you want to check, in a program executing unit tests, that function `Sqrt` has a precondition detecting negative inputs. This assumes the program is compiled in **Eval_and_abort** translation mode.

```

auto test_precondition_of_sqrt()
{
    bool function_finished = false;

    std::abortable_component([&]{
        (void)Sqrt(-1.0);
        function_finished = true;
    });

    EXPECT(!function_finished);
}

```

4.1.4. Resetting the program

{dis.use.res}

Suppose that after a detected failure, you want to restart the program, but from within the program.

```

int main()
{
    bool finished = false;

    std::abortable_component([&]{
        finished = main_program_loop();
    });

    while (!finished)
    {
        std::abortable_component([&]{
            necessary_critical_cleanup();
            finished = main_program_loop();
        });
    }
}

```

5. Conclusion

{con}

In this paper we demonstrated that there is at least one way to add support for not-aborting the program upon detecting contract violation (while at the same time not letting the code dependent on the declared contract execute) after the MVP, while retaining backwards compatibility. "After the MVP" may mean "still in the same release cycle". This solution does not require the introduction of a new translation mode. Other ways of addressing the same problem, also suitable for a post-MVP addition, are also possible; for instance, the ability to install a custom violation handler where throwing an exception is one of the options.

7. References

{ref}

Joshua Berne has reviewed this paper and contributed to its quality.

8. References

{ref}

- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P2388R4] — Andrzej Krzemiński, Gašper Ažman, "Minimum Contract Support: either *No_eval* or *Eval_and_abort*" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2388r4.html>).
- [P2698R0] — Bjarne Stroustrup, "Unconditional termination is a serious problem" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2698r0.pdf>).